

IA02 : Classification non linéaire grâce à l'IA avancée

Arnaud Godard, Gatien Genevois

Année 2025 - 2026



L'objectif de ce projet était de créer plusieurs algorithmes de classification non linéaires grâce à l'IA avancée.

Le projet se décompose en deux parties. La première concerne la création de plusieurs algorithmes de classification multiclasse à partir du jeu de données CIFAR-10, dans lequel les images sont étiquetées en dix classes : avion, voiture, oiseau, chat, cerf, chien, grenouille, cheval, bateau et camion.

Il faudra créer :

- deux algorithmes de machine learning classiques au choix ;
- un CNN avec trois architectures différentes au choix ;
- un CNN hybride au choix.

La deuxième partie concerne la création de trois types d'algorithmes d'IA pour la classification binaire d'images de fractures issues du jeu de données « Fracture Classification Dataset ». L'objectif est de détecter s'il y a ou non une fracture sur l'image. Il faudra également augmenter les données afin d'essayer d'améliorer les performances des différents modèles.



Partie 1: Classification multiclasse du dataset CIFAR-10

La première étape a consisté à préparer les données. Le jeu de données a donc été séparé en trois ensembles : un ensemble d'entraînement, un ensemble de test et un ensemble de validation. Les images ont ensuite été normalisées, puis les labels ont été encodés.

```
# load CIFAR-10 dataset
(x_train_T, y_train_T), (x_test, y_test) = tf.keras.datasets.cifar10.load_data()

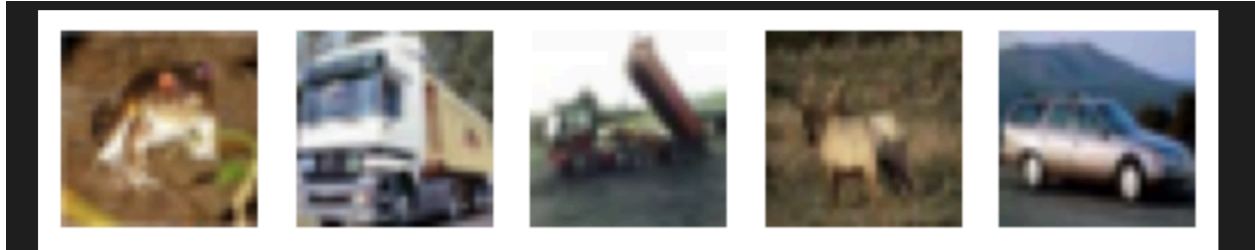
N_train_T = np.shape(x_train_T)[0]
N_train = round(0.8*N_train_T) # 80% for train and 20% for validation
x_train, y_train = x_train_T[0:N_train], y_train_T[0:N_train]
x_val, y_val = x_train_T[N_train:], y_train_T[N_train:]

# Pixels normalization ([0,255] to [0,1])
x_train, x_val, x_test = x_train / 255.0, x_val / 255.0, x_test / 255.0

# One-Hot encoding
y_train_c = to_categorical(y_train, 10)
y_val_c = to_categorical(y_val, 10)
y_test_c = to_categorical(y_test, 10)
```

La visualisation des images a également permis de mieux comprendre les données manipulées.

```
for i in range(5):
    plt.subplot(1,5,i+1)
    plt.imshow(x_train[i], cmap='gray')
    plt.axis('off')
plt.show()
```



Pour les deux modèles de machine learning classiques, nous avons choisi de réaliser un SVM ainsi qu'un KNN.

Avant cela, nous avons appliqué une PCA afin de calculer les vecteurs propres et les valeurs propres de la matrice de covariance des données. Les données transformées ont ensuite été utilisées pour entraîner nos modèles.

```
pca_model = PCA(n_components=1)
x_train = x_train.reshape(x_train.shape[0], -1)
x_test = x_test.reshape(x_test.shape[0], -1)
x_train_pca = pca_model.fit_transform(x_train)
x_test_pca = pca_model.transform(x_test)
```

Pour trouver les meilleurs paramètres, nous avons utilisé GridSearchCV pour le SVM et le KNN.

Les meilleurs paramètres trouvés pour le SVM sont :

```
Best parameters: {'C': 100, 'gamma': 0.001, 'kernel': 'rbf'}
Best cross-validation accuracy: 0.16440004813248268
Test accuracy: 0.1622
Training time: 6362.221035957336 seconds
```

Et pour le KNN:

```
Best parameters: {'metric': 'euclidean', 'n_neighbors': 21, 'weights': 'uniform'}
Best cross-validation accuracy: 0.13687999999999997
Test accuracy: 0.1435
Training time: 2.710460901260376 seconds
```

Pour les deux meilleurs modèles, l'accuracy est de 0,16 pour le SVM et de 0,14 pour le KNN, avec un temps d'entraînement de 66 secondes. L'accuracy est donc très faible.

Par la suite, nous avons créé un CNN avec trois architectures différentes.

Présentation des 3 CNN

CNN Basique

```
model_basic = Sequential([
    Conv2D(32, (3,3), activation='relu',
        input_shape=(32,32,3),
        padding='same'),
    MaxPooling2D(2,2),
    Conv2D(64, (3,3), activation='relu',
        padding='same'),
    MaxPooling2D(2,2),
    Flatten(),
    Dense(128, activation='relu'),
    Dense(10, activation='softmax')
])
```



Architecture simple avec 2 blocs
convolution/pooling

CNN Intermédiaire

```
model_medium = Sequential([
    Conv2D(32, (3,3), activation='relu', input_shape=(32,32,3),
        padding='same'),
    BatchNormalization(),
    Conv2D(32, (3,3), activation='relu', input_shape=(32,32,3),
        padding='same'),
    BatchNormalization(),
    MaxPooling2D(2,2),
    Dropout(0.25),
    Conv2D(64, (3,3), activation='relu',padding='same'),
    BatchNormalization(),
    Conv2D(64, (3,3), activation='relu',padding='same'),
    BatchNormalization(),
    MaxPooling2D(2,2),
    Dropout(0.25),
    Flatten(),
    Dense(128, activation='relu'),
    Dropout(0.25),
    Dense(10, activation='softmax')
])
```



Ajout de BatchNorm et Dropout pour
une meilleure régularisation

CNN Profond

```
model_deep = Sequential([
    Conv2D(64, (3,3), activation='relu', input_shape=(32,32,3),
        padding='same'),
    Conv2D(64, (3,3), activation='relu', input_shape=(32,32,3),
        padding='same'),
    Conv2D(128, (3,3), activation='relu', input_shape=(32,32,3),
        padding='same'),
    Conv2D(128, (3,3), activation='relu', input_shape=(32,32,3),
        padding='same'),
    MaxPooling2D(2,2),
    Conv2D(256, (3,3), activation='relu', input_shape=(32,32,3),
        padding='same'),
    Conv2D(256, (3,3), activation='relu', input_shape=(32,32,3),
        padding='same'),
    Conv2D(256, (3,3), activation='relu', input_shape=(32,32,3),
        padding='same'),
    MaxPooling2D(2,2),
    Flatten(),
    Dense(128, activation='relu'),
    Dropout(0.25),
    Dense(10, activation='softmax')
])
```



Architecture plus profonde avec
64, 128 et 256 filtres

L'idée principale était d'utiliser des CNN avec des architectures de plus en plus complexes. Le premier CNN est plutôt simple, avec deux blocs de convolution et de pooling. Le deuxième ajoute de la batch normalization ainsi que du dropout pour la régularisation. Le dernier CNN est le plus profond, avec 64, 128 puis 256 filtres.

Pour chaque CNN, nous avons obtenu les mesures suivantes :

 Tableau comparatif des 3 CNN Performances, temps d'entraînement et temps de test 								
Modèle	Accuracy entraînement	Loss entraînement	Val Accuracy	Val Loss	Test Accuracy	Training Time	Test Time	Temps/step
 CNN Basique	0.8176	0.5174	0.6982	0.9462	0.6872	87.0 s	0.8 s	9 ms
 CNN Intermédiaire	0.6488	1.0041	0.6649	0.9612	0.5268	123.2 s	2.3 s	32 ms
 CNN Profond	0.7918	0.5852	0.7314	0.8136	0.7146	4104.3 s	30.0 s	411 ms



LECTURE RAPIDE

-  **CNN Basique** : meilleur compromis rapidité / performance
-  **CNN Intermédiaire** : performances les plus faibles sur cette expérience
-  **CNN Profond** : meilleure précision, mais coût temporel très élevé

Le CNN basique est le plus rapide à entraîner, avec seulement 87 secondes, et il obtient une bonne accuracy de test de 68,72 %.

Le CNN intermédiaire est moins performant ici, avec une accuracy de test de seulement 52,68 %, malgré l'ajout de BatchNormalization et de Dropout.

Le CNN profond obtient les meilleurs résultats en validation et en test, avec une test accuracy de 71,46 %. Cependant, il est beaucoup plus coûteux en temps : 4 104,3 secondes, soit environ 47 fois plus lent que le CNN basique.

Le CNN profond est donc le plus performant, mais le CNN basique offre le meilleur compromis entre performance et rapidité. Le CNN intermédiaire semble sous-performer dans cette expérience.

Pour la création du CNN hybride, nous avons choisi de créer un modèle reprenant la partie convolutionnelle du modèle profond créé précédemment, associée au SVM avec les meilleurs paramètres trouvés précédemment pour la partie classification.

Pour cela, nous avons chargé les poids du modèle profond, puis retiré la dernière couche softmax à l'aide de la méthode `model_deep.pop()`, afin d'utiliser le modèle comme extracteur de caractéristiques pour le SVM.

```
# load model weights from the previous training
model_deep.load_weights('models_weight/model_deep.weights.h5')
model_deep.summary()
```

Ce CNN hybride nous donne une accuracy de 0,72.

Partie 2 : Prédiction de fracture

1/ Exploration des données

Les données sont composées de plusieurs fichiers et dossiers :

- un fichier "dataset.csv" présentant les métadonnées du jeu de données.
- un dossier "Image" composé de deux sous-dossiers "Fractured" et "Non_Fractured" qui contiennent les images réparties par classe.

Le dataset ne présente que des métadonnées sur les différentes photos :

On remarque donc que le jeu de données est composé de 4083 images, dont 717 catégorisées comme représentant une fracture.

Il a donc fallu récupérer ces images pour pouvoir les traiter avec les métadonnées, en allant les chercher dans des dossiers différents selon la catégorie.

2/ Les 3 Modèles utilisés :

1/ HOG et SVM

Le HOG (Histogram of Oriented Gradients) est un extracteur de features de scikit-image (bibliothèque créée pour le traitement d'image) . Il était beaucoup utilisé (d'après nos recherches) avant l'arrivée des modèles de Deep Learning. Nous avons donc choisi de le combiner avec un modèle SVM.

Pour l'optimisation du modèle nous avons testé différents noyaux (gaussien et polynomial) pour une tolérance de l'erreur fixée à 1.0 (C). Pour évaluer l'efficacité du modèle. Nous avons étudié l'accuracy, les f1-score et le recall des différentes classes grâce au "classification report" de sklearn.metrics.

Voici les résultats :

Noyau = Poly	Precision	Recall	F1-score	support
Sain	0.88	0.97	0.92	662
Fracture	0.76	0.37	0.50	143
Accuracy			0.87	805
Macro avg	0.82	0.67	0.71	805
Weighted avg	0.86	0.87	0.85	805

Noyau = RBF	Precision	Recall	F1-score	support
Sain	0.84	1.00	0.91	662
Fracture	0.86	0.13	0.23	143
Accuracy			0.84	805
Macro avg	0.85	0.56	0.57	805
Weighted avg	0.85	0.84	0.79	805

L'étude nous montre que le RBF représente une accuracy importante pour la détection des sains. Mais on remarque que le recall et le F1-score des Fractures sont très bas. On peut en déduire que le modèle a appris à prédire principalement une radio saine, au vu du nombre important de radio saines par proportion. Ce modèle semble alors plutôt mauvais.

Le SVM à noyau polynomial améliore ces deux facteurs en plus de l'accuracy globale. Ce modèle semble donc davantage généralisable mais un recall de 0.37 reste très faible, surtout dans un domaine médical où il serait mieux de donner des faux positifs que de faux négatifs.

Nous avons donc tenté de trouver les paramètres les plus optimisés pour créer un SVM à noyau polynomial. Pour cela nous avons utilisé GridSearchCV de sklearn.model_selection. Il a défini le modèle le plus adapté en prenant en compte le f1_score et en optimisant les paramètres du degré et la tolérance à l'erreur. De plus nous avons rajouté le paramètre : class_weight='balanced' pour ajuster les poids en prenant en compte la proportion des classes dans le dataset.

```
from sklearn.model_selection import GridSearchCV

base_svm = SVC(kernel='poly', class_weight='balanced', random_state=42)

param_grid = {'degree': [2, 3, 4], 'C': [0.1, 1, 10]}

print("Recherche des meilleurs hyperparamètres en cours (Grid Search)...")
grid_search = GridSearchCV(estimator=base_svm, param_grid=param_grid, cv=5, scoring='f1_macro', n_jobs=-1)
grid_search.fit(X_train, y_train)

print("OPTIMISATION TERMINÉE")
print(f"Meilleurs paramètres trouvés : {grid_search.best_params_}")
print(f"Meilleur score de validation (F1-macro) : {grid_search.best_score_ * 100:.2f}%")

best_svm_model = grid_search.best_estimator_
y_pred = best_svm_model.predict(X_test)

print("RÉSULTATS DU MEILLEUR SVM SUR LE JEU DE TEST")
print(classification_report(y_test, y_pred, target_names=['Sain', 'Fracture']))
```

```
Recherche des meilleurs hyperparamètres en cours (Grid Search)...
OPTIMISATION TERMINÉE
Meilleurs paramètres trouvés : {'C': 1, 'degree': 3}
Meilleur score de validation (F1-macro) : 71.64%
RÉSULTATS DU MEILLEUR SVM SUR LE JEU DE TEST
```

	precision	recall	f1-score	support
Sain	0.90	0.92	0.91	662
Fracture	0.60	0.55	0.57	143
accuracy			0.85	805
macro avg	0.75	0.73	0.74	805
weighted avg	0.85	0.85	0.85	805

Ceci est donc le meilleur rendu de cette combinaison HOG +SVM ce qui n'est pas le plus efficace.

2/ CNN

On reprend les données on les split en train (80%), test (20%). Puis nous utilisons un split pour créer les valeurs de validation pour les époques grâce à un split de 15% des valeurs de train.

```
plt.figure
for i in range(5):
    plt.subplot(1,5,i+1)
    plt.imshow(X_train[i],cmap='gray')
    plt.axis('off')
plt.show()

print("Séparation des données en train/test...")
X_train, X_test, y_train, y_test = train_test_split(
    X_raw_images, y_labels, test_size=0.2, random_state=42, stratify=y_labels
)

print("Création des valeurs de test par époque...")
X_train_reel, X_val, y_train_reel, y_val_c = train_test_split(
    X_train, y_train, test_size=0.15, random_state=42, stratify=y_train
)

print(f"Entraînement (X_train) : {X_train.shape}")
print(f"Validation (X_val) : {X_val.shape}")
print(f"Test (X_test) : {X_test.shape}")
```



```
Séparation des données en train/test...
Création des valeurs de test par époque...
Entraînement (X_train) : (3219, 128, 128, 1)
Validation (X_val) : (483, 128, 128, 1)
Test (X_test) : (805, 128, 128, 1)
```

Après de nombreux essais de création du CNN deux modèles se sont révélés être les meilleurs :

- un qui a maximisé l'accuracy avec des résultats extrêmement similaires au modèle de HOG +SVM
- un second qui lui a maximisé le recall des fractures

Nous avons donc choisi, compte tenu du sujet et de l'importance de prédire un minimum de faux négatifs, nous avons choisi le second.

Voici sa structure :

```
model = models.Sequential([
    layers.Conv2D(32, (3, 3), activation='relu', input_shape=IMG_SHAPE),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Dropout(0.2),
    layers.Conv2D(128, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Dropout(0.2),

    layers.Flatten(),
    layers.Dense(128, activation='relu'),
    layers.Dropout(0.5),

    layers.Dense(1, activation='sigmoid')
])

model.compile(optimizer='adam', loss='binary_crossentropy',
              metrics=['accuracy', tf.keras.metrics.Recall(name='recall')])
```

Il est composé de 3 couches de convolutions de fonction d'activation Relu avec des Maxpooling 2,2. Des Dropouts de 20% sont appliqués aux deux dernières couches de convolutions.

Une couche dense de 128 neurones est également ajoutée avec un dropout plus important pour améliorer la généralisation.

Enfin la couche de sortie est donnée par une fonction d'activation sigmoid pour une classification binaire.

L'entraînement s'est effectué notamment avec un early stopping sur la validation recall et en tentant de prendre en compte la répartition inégale du dataset par la définition du class_weight en balanced. L'entraînement initial s'est effectué sur 15 époques et avec un batch_size de 32.

Après entraînement et évaluation voici les résultats :

	precision	recall	f1-score	support
0	0.90	0.48	0.62	662
1	0.24	0.76	0.36	143
accuracy			0.53	805
macro avg	0.57	0.62	0.49	805
weighted avg	0.79	0.53	0.58	805

Training Time : 105.8 secondes	Test time : 1.3 secondes
--------------------------------	--------------------------

On remarque que le modèle prédit juste 76 % des fractures ce qui est un très bon point pour notre cas, cependant il a un taux de 50% de faux positif (plus d'une image saine sur 2 est prédite comme fracturée). Ainsi ce modèle a bien compris l'importance de ne pas manquer les fractures, mais le fait au détriment de l'accuracy sur les images non fracturées.

L'utilisation d'un modèle hybride pourrait alors améliorer la précision de la prédiction, maintenant que les features importantes des images semblent être maîtrisées.

3/ CNN hybride

On réutilise le CNN et on rajoute un SVM à la place de la couche de sortie. Puis on le test voici la sortie :

Layer (type)	Output Shape	Param #
conv2d_32 (Conv2D)	(None, 128, 128, 32)	320
max_pooling2d_31 (MaxPooling2D)	(None, 63, 63, 32)	0
conv2d_33 (Conv2D)	(None, 61, 61, 64)	18,496
max_pooling2d_32 (MaxPooling2D)	(None, 30, 30, 64)	0
dropout_24 (Dropout)	(None, 30, 30, 64)	0
conv2d_34 (Conv2D)	(None, 28, 28, 128)	73,856

OPTIMISATION TERMINÉE				
Meilleurs paramètres trouvés : {'C': 0.1, 'degree': 4, 'kernel': 'poly'}				
Test du modèle hybride optimisé....				
	precision	recall	f1-score	support
Sain	0.87	0.83	0.85	662
Fracture	0.35	0.43	0.39	143
accuracy			0.76	805
macro avg	0.61	0.63	0.62	805
weighted avg	0.78	0.76	0.77	805

On se retrouve alors avec un modèle qui retrouve une accuracy correcte et un recall des images de fracture qui se rapproche des 50% mais qui reste moins performant au final que les 2 autres modèles puisqu'il n'a ni l'accuracy du premier ni le recall du deuxième.

Les principaux problèmes de performances de ces différents modèles sont avant tout liés au déséquilibre du dataset, une solution pour les optimiser est donc d'interagir directement avec le dataset pour augmenter les données d'images représentant des fractures

3/ L'augmentation de donnée

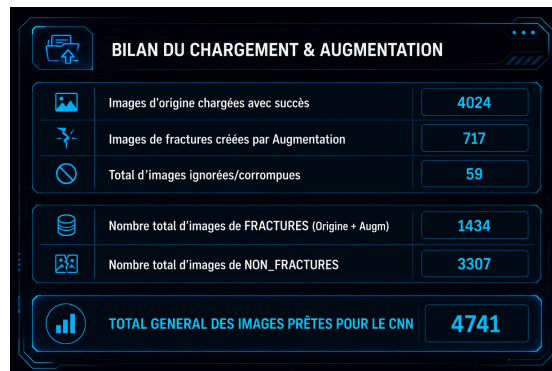
1/ Protocole d'augmentation de donnée :

Pour cette deuxième partie, nous avons décidé de créer un nouveau notebook, et nous effectuons l'augmentation lors de la récupération des images.

Nous avons décidé d'augmenter uniquement les images de fracture pour équilibrer davantage le dataset.

Pour cela nous avons donc choisi d'utiliser des librairies Numpy, et skimage.transform notamment. Nous avons effectué des flips (horizontaux et verticaux), des rotations (avec un angle de -15 à 15 degrés) ainsi que des translations légères de -10 à 10 pixels.

Ainsi voici le résumé du dataset final utilisé après l'augmentation des données :



Images d'origine chargées avec succès	4024
Images de fractures créées par Augmentation	717
Total d'images ignorées/corrompues	59
Nombre total d'images de FRACTURES (Origine + Augm)	1434
Nombre total d'images de NON_FRACTURES	3307
TOTAL GENERAL DES IMAGES PRÊTES POUR LE CNN	4741

Le nombre d'images fracturées est donc doublé au final ce qui permet donc au modèles de ne pas sur-apprendre sur les images non fracturées.

Justement entraînons et évaluons à nouveau les modèles précédents avec ce nouveau dataset augmenté :

2/ Pour le modèle d' HOG +SVM (optimisé par l'algorithme de Grid Search):

```
RÉSULTATS DU GRID SEARCH
Meilleurs hyperparamètres trouvés : {'C': 10, 'degree': 2, 'kernel': 'rbf'}
Meilleur score en validation croisée : 85.05%

RÉSULTATS DU MODÈLE OPTIMISÉ HOG + SVM (SUR DATASET AUGMENTÉ)
```

	precision	recall	f1-score	support
Sain	0.88	0.94	0.91	662
Fracture	0.83	0.71	0.76	287
accuracy			0.87	949
macro avg	0.85	0.82	0.84	949
weighted avg	0.86	0.87	0.86	949

La différence avec l'ancien modèle est flagrante ! L'accuracy est légèrement supérieure : $0.87 > 0.85$. Mais l'important est surtout au niveau du recall de fracture puisque celui-ci dépasse les 70% ! Ce qui est primordial pour la prédiction de fracture sur l'imagerie médicale.

3/ Pour le modèle de CNN classique :

30/30	1s 44ms/step			
	precision	recall	f1-score	support
0	0.86	0.87	0.86	662
1	0.68	0.66	0.67	287
accuracy			0.80	949
macro avg	0.77	0.76	0.77	949
weighted avg	0.80	0.80	0.80	949
Training Time: 286.7 seconds, Test time: 1.6 seconds				

Là encore nous remarquons une différence avec l'ancien modèle : nous avons certes perdu 10% de recall mais nous avons eu une augmentation d'accuracy conséquente de plus de 30% ! Ce modèle augmenté reste néanmoins moins bon que celui qui utilise le SVM (accuracy et recall plus bas)

4/ Pour le modèle de CNN hybride :

```
Extraction des features du CNN...
119/119 _____ 5s 39ms/step
30/30 _____ 1s 41ms/step

Recherche des meilleurs hyperparamètres pour le SVM hybride...

OPTIMISATION TERMINÉE
Meilleurs paramètres trouvés : {'C': 1, 'degree': 3, 'kernel': 'poly'}

Test du modèle hybride optimisé...
      precision    recall  f1-score   support

   Sain           0.86      0.86      0.86       662
  Fracture        0.68      0.68      0.68       287

 accuracy          0.81          0.81          0.81       949
 macro avg         0.77      0.77      0.77       949
weighted avg         0.81      0.81      0.81       949
```

De nouveau si nous comparons les performances des modèles avant et après augmentation on remarque un écart flagrant : Toutes les données de vérification sont augmentées.

Finalement le modèle le plus performant est le modèle combinant le HOG et le SVM ajusté avec le dataset augmenté.

Les principales difficultés rencontrées :

Concernant la prédiction des fractures, de nombreuses expérimentations ont été réalisées afin d'optimiser le modèle CNN, notamment en ajustant différents hyperparamètres tels que le nombre d'époques, le taux de dropout, l'early stopping ou encore l'ajout de couches de Batch Normalization. Malgré ces ajustements, les résultats sont restés limités. En raison du fort déséquilibre entre les classes du dataset, le modèle avait tendance à prédire majoritairement la classe *non_fracture*, ce qui augmentait artificiellement l'accuracy. Plusieurs solutions ont été testées pour corriger ce biais, notamment l'optimisation du recall et du score F1 ainsi que l'utilisation de poids de classes, sans amélioration significative des performances.

Enfin, lors de la récupération et du chargement des données, 59 images ont été détectées comme corrompues et ont dû être exclues du dataset.